

Compte rendu TP3 :

DNS(Domain Name System)

DNS traduit les noms de domaine en adresses IP et inversement. C'est comme un annuaire d'Internet. Par exemple, lorsqu'un utilisateur saisit une URL dans son navigateur, le client DNS interroge un serveur DNS pour connaître l'adresse IP associée. Le navigateur se connecte ensuite au serveur web via cette IP. Le serveur DNS écoute sur le port 53 (au niveau de la couche transport).

1- Lancer et capturer les résultats de la commande
nslookup iut - rt . univ - artois . Fr

-Encapsulation :
ETH | IP | UDP | DNS

-ETH :
@MAC de destination : D0 E0 42 80 14 55
@MAC source : CC 96 E5 1C 34 D6

IP :
@IP source : 172.31.25.145
@IP de destination : 172.18.26.101

UDP :
Port source : 45192
Port de destination : 53

DNS(query) : iut-rt.univ-artois.fr

-ETH :
@MAC de destination : CC 96 E5 1C 34 D6
@MAC source : D0 E0 42 80 14 55

IP :
@IP source : 172.18.26.101
@IP de destination : 172.31.25.145

UDP :
Port source : 53
Port de destination : 45192

DNS(response) : iut-rt.univ-artois.fr
DNS(response) : iutb-etu-gtr-25-9.univ-artois.fr
DNS(response) : Adresse : 172.31.25.9

-ETH :

@MAC de destination : D0 E0 42 80 14 55

@MAC source : CC 96 E5 1C 34 D6

IP :

@IP source : 172.31.25.145

@IP de destination : 172.18.26.101

UDP :

Port source : 45192

Port de destination : 53

DNS(query) : iutb-etu-gtr-25-9.univ-artois.fr

-ETH :

@MAC de destination : CC 96 E5 1C 34 D6

@MAC source : D0 E0 42 80 14 55

IP :

@IP source : 172.18.26.101

@IP de destination : 172.31.25.145

UDP :

Port source : 53

Port de destination : 45192

DNS(response) : iutb-etu-gtr-25-9.univ-artois.fr(name)

DNS(response) : ns.univ-artois.fr(Primary name)

2.Réaliser la même analyse avec la commande suivante
nslookup 172.31.25.9

-Encapsulation :

ETH | IP | UDP | DNS

-ETH :

@MAC de destination : D0 E0 42 80 14 55

@MAC source : CC 96 E5 1C 34 D6

IP :

@IP source : 172.31.25.145

@IP de destination : 172.18.26.101

UDP :

Port source : 43701

Port de destination : 53

DNS(query) : 9.25.31.172.in-addr.arpa

-ETH :

@MAC de destination : CC 96 E5 1C 34 D6

@MAC source : D0 E0 42 80 14 55

IP :

@IP source : 172.18.26.101

@IP de destination : 172.31.25.145

UDP :

Port source : 53

Port de destination : 43701

DNS(response) : 9.25.31.172.in-addr.arpa

DNS(response) : iut-rt.edu.ua

DHCP(Dinamyc Host Control Protocol)

Un serveur DHCP attribue automatiquement une configuration IP (@IP, passerelle, DNS, masque de sous-réseau) à un appareil. Étapes :

1. Découverte (Discover) : le client demande une IP.
2. Offre (Offer) : le serveur propose une IP.
3. Requête (Request) : le client accepte.
4. Accusé (Acknowledge) : le serveur confirme l'attribution.

Le serveur DHCP écoute sur le port 67 (au niveau de la couche transport) et le client émet depuis son port 68.

1. Analyse des trames dans le fichier dhcp.pcap

2. Ces échanges ont pour but l'attribution des adresses IP à un hôte au sein d'un réseau

3. Analyse des trames :

ETH | IP | UDP | DHCP

-ETH :

@MAC de destination : FF FF FF FF FF FF

@MAC source : 00 0B 82 01 FC 42

IP :

@IP source : 0.0.0.0

@IP de destination : 255.255.255.255

UDP :

Port source : 68

Port de destination : 67

DHCP (Discover)

*ETH :

@MAC de destination : 00 0B 82 01 FC 42

@MAC source : 00 08 74 AD F1 9B

IP :

@IP source : 192.168.0.1

@IP de destination : 192.168.0.10

UDP :

Port source : 67

Port de destination : 68

DHCP (offer)

*ETH :

@MAC de destination : FF FF FF FF FF FF

@MAC source : 00 0B 82 01 FC 42

IP :

@IP source : 0.0.0.0

@IP de destination : 255.255.255.255

UDP :

Port source : 68

Port de destination : 67

DHCP (Discover)

*ETH :

@MAC de destination : FF FF FF FF FF FF

@MAC source : 00 0B 82 01 FC 42

IP :

@IP source : 0.0.0.0

@IP de destination : 255.255.255.255

UDP :

Port source : 68

Port de destination : 67

DHCP(request)

*ETH :

@MAC de destination : 00 0B 82 01 FC 42

@MAC source : 00 08 74 AD F1 9B

IP :

@IP source : 192.168.0.1

@IP de destination : 192.168.0.10

UDP :

Port source : 67

Port de destination : 68

DHCP(ACK)

SSH(Secure Shell)

SSH permet une connexion sécurisée à distance à un autre équipement. Il est très utilisé pour l'administration de systèmes (serveurs, ...) et d'équipements réseaux (routeurs, switchs, ...). Il fonctionne avec un chiffrement des échanges entre le client et le serveur. Le serveur SSH écoute sur le port 22 (au niveau de la couche transport).

1. Capture des trames issues de cette commande ssh iut-rt (Lancer d'abord la capture Wireshark puis lancer la commande, loguez-vous avec vos identifiants, puis taper la commande ls, puis arrêter la capture)

2. Ces échanges ont pour but d'effectuer une connexion sécurisée à distance à un autre équipement. Ces échanges sont chiffrés.

3.

ETH | IP | TCP | SSH

*ETH :

@MAC de destination : 2A 91 45 2F A3 A9

@MAC source : CC 96 E5 1C 34 D6

01 80 C2 00 00 03 00 14 22 E9 54 5E 88 8E 01 00 00 22 02 01 00 22 04 10 3A 53 5F 0E E8 C6 B3
4F E7 14 AA 7D AD 9A 0E 15 4A 6F 68 6E 2E 4D 63 47 75 69 72 6B

IP :

@IP source : 172.31.25.145

@IP de destination : 172.31.25.9

TCP :

Port source : 50010

Port de destination : 22

ACK : 1

SSH : Ubuntu-3ubuntu13.13

*ETH :

@MAC de destination : CC 96 E5 1C 34 D6

@MAC source : 2A 91 45 2F A3 A9

IP :

@IP source : 172.31.25.9

@IP de destination : 172.31.25.145

TCP :

Port source : 22

Port de destination : 50010
ACK : 44
SSH : Ubuntu-3ubuntu13.13

*ETH :
@MAC de destination : 2A 91 45 2F A3 A9
@MAC source : CC 96 E5 1C 34 D6

IP :
@IP source : 172.31.25.145
@IP de destination : 172.31.25.9

TCP :
Port source : 50010
Port de destination : 22
ACK :43
SSH : Encrypted packet

*ETH :
@MAC de destination : 2A 91 45 2F A3 A9
@MAC source : CC 96 E5 1C 34 D6

IP :
@IP source : 172.31.25.145
@IP de destination : 172.31.25.9

TCP :
Port source : 50010
Port de destination : 22
ACK :43
SSH : Encrypted packet

*ETH :
@MAC de destination : CC 96 E5 1C 34 D6
@MAC source : 2A 91 45 2F A3 A9

IP :
@IP source : 172.31.25.9
@IP de destination : 172.31.25.145

TCP :
Port source : 22
Port de destination : 50010
ACK : 1580
SSH : Encrypted packet

*ETH :
@MAC de destination : CC 96 E5 1C 34 D6
@MAC source : 2A 91 45 2F A3 A9

IP :
@IP source : 172.31.25.9
@IP de destination : 172.31.25.145

TCP :
Port source : 22
Port de destination : 50010
ACK :2788
SSH : Encrypted packet

*ETH :
@MAC de destination : CC 96 E5 1C 34 D6
@MAC source : 2A 91 45 2F A3 A9

IP :
@IP source : 172.31.25.9
@IP de destination : 172.31.25.145

TCP :
Port source : 22
Port de destination : 50010
ACK :2788
SSH : Encrypted packet

FTP(File Transfert Protocol)

FTP sert à transférer des fichiers entre un client et un serveur. Il utilise deux ports : le 21 pour le contrôle, et le 20 pour les données. Attention FTP n'est pas chiffré aussi les logins et mots de passe passent en clair.

1.après les trames issues de cette commande ftp iut-rt (loguez-vous avec le login guest00 et le mot de passe progr00, puis taper la commande ls, puis taper quit puis arrêter la capture) (Lancer d'abord la capture Wireshark puis lancer la commande)

2.Ces échanges servent à transférer des fichiers entre un client et un serveur

3.Analyse des échanges

*ETH :
@MAC de destination : CC 96 E5 1C 34 D6
@MAC source : 2A 91 45 2F A3 A9

IP :
@IP source : 172.31.25.9
@IP de destination : 172.31.25.145

TCP :

Port source : 21
Port de destination : 57202
ACK : 1

FTP:
220

*ETH :
@MAC de destination : 2A 91 45 2F A3 A9
@MAC source : CC 96 E5 1C 34 D6

IP :
@IP source : 172.31.25.145
@IP de destination : 172.31.25.9

TCP :
Port source : 57202
Port de destination : 21
ACK : 21

FTP:

*ETH :
@MAC de destination : CC 96 E5 1C 34 D6
@MAC source : 2A 91 45 2F A3 A9

IP :
@IP source : 172.31.25.9
@IP de destination : 172.31.25.145

TCP :
Port source : 21
Port de destination : 57202
ACK : 35

FTP:
Response code : 331

*ETH :
@MAC de destination : 2A 91 45 2F A3 A9
@MAC source : CC 96 E5 1C 34 D6

IP :
@IP source : 172.31.25.145
@IP de destination : 172.31.25.9

TCP :
Port source : 57202
Port de destination : 21
ACK : 55

*ETH :

@MAC de destination : CC 96 E5 1C 34 D6

@MAC source : 2A 91 45 2F A3 A9

IP :

@IP source : 172.31.25.9

@IP de destination : 172.31.25.145

TCP :

Port source : 21

Port de destination : 57202

ACK : 35

FTP:

Response code : 331

*ETH :

@MAC de destination : CC 96 E5 1C 34 D6

@MAC source : 2A 91 45 2F A3 A9

IP :

@IP source : 172.31.25.9

@IP de destination : 172.31.25.145

TCP :

Port source : 21

Port de destination : 57202

ACK : 35

FTP:

Response code : 230

(Analyse non terminée)

-clic droit sur un des échanges FTP puis Suivre
| flux TCP (ou follow | TCP Stream).

```
USER ftp iut-rt
331 Please specify the password.
PASS R3g1sd1az#
530 Login incorrect.
QUIT
221 Goodbye.
```

HTTP (HyperText Transfer Protocol)

HTTP permet le transfert de données web (HTML, images, vidéos, ...) sur un navigateur. HTTP utilise un modèle de requête/réponse. Le client peut envoyer une requête (GET, POST, PUT, ...) et le serveur renvoie une réponse (page HTML, codes, ...). Un serveur web HTTP

écoute sur le port 80. La version sécurisée de HTTP est HTTPS (ajout d'une couche de chiffrement TLS - Transport Layer Security). Un serveur HTTPS écoute sur le port 443.

1. Capturer les trames issues de votre connexion depuis un navigateur au site <http://iut-rt.univ-artois.fr>
2. Ces échanges ont pour but de charger les informations d'une page web sur un navigateur
3. Dessiner leur encapsulation, et écrire les adresses MAC, IP et ports mis en jeu.

ETH | IP | TCP | HTTP

@MAC dst: 2a:91:45:2f:a3:a9
@MAC src :8c:ec:4b:83:bc:cc
@IP src : 172.31.25.26
@IP dst : 172.31.25.9
Port src :54062
Port dst : 80

@MAC dst: 8c:ec:4b:83:bc:cc
@MAC src :2a:91:45:2f:a3:a9
@IP src : 172.31.25.9
@IP dst : 172.31.25.26
Port src : 80
Port dst : 54062

@MAC dst: 2a:91:45:2f:a3:a9
@MAC src :8c:ec:4b:83:bc:cc
@IP src : 172.31.25.26
@IP dst : 172.31.25.9
Port src :54062
Port dst : 80

@MAC dst: 8c:ec:4b:83:bc:cc
@MAC src :2a:91:45:2f:a3:a9
@IP src : 172.31.25.9
@IP dst : 172.31.25.26
Port src : 80
Port dst : 54062

4. Faire un clic droit sur un échange HTTP puis Suivre ou Follow | TCP Stream. Analyser ces échanges (étudier le contenu des en-têtes HTTP).

```
GET / HTTP/1.1
Host: iut-rt.univ-artois.fr
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i

HTTP/1.1 200 OK
Date: Tue, 14 Oct 2025 10:15:03 GMT
Server: Apache/2.4.52 (Ubuntu)
Vary: Cookie,Accept-Encoding
Set-Cookie: DokuWiki=kbvemv2cepnh08d66plj18ml97; path=/wiki/; HttpOnly
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: DWcb100c0f811c1b6d691e3e7c406b56ba=deleted; expires=Thu, 01-Jan-1970 00:00:01 GMT
; Max-Age=0; path=/wiki/; HttpOnly
Content-Encoding: gzip
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html; charset=utf-8
```

-Requête

Ligne de requête : GET / HTTP/1.1, Le client demande la ressource racine (/) de l'hôte, en utilisant le protocole HTTP version 1.1.

Hôte : Host : iut-rt.univ-artois.fr, L'adresse du serveur ciblé.

Identifiant client : Mozilla/5.0...Le client est le navigateur Firefox 128.0, fonctionnant sur Linux.

Négociation : Accepte-Encoding : gzip, deflate Le client est capable de recevoir du contenu compressé (pour réduire le temps de transfert)

Gestion de la connexion : Connection : keep-alive Le client demande à maintenir la connexion ouverte après cette transaction pour d'éventuelles requêtes futures.

-Réponse :

HTTP/1.1 200 OK : Confirme que la requête a été traitée avec succès et que la ressource est fournie

Server: Apache/2.4.52 (Ubuntu) : Le serveur est de type Apache, version 2.4.52

Vary: Cookie,Accept-Encoding

Set-Cookie: DokuWiki=kbvemv2cepnh08d66plj18ml97; path=/wiki/; HttpOnly

Expires: Thu, 19 Nov 1981 08:52:00 GMT

Cache-Control: no-store, no-cache, must-revalidate : Le serveur interdit formellement au client et aux intermédiaires de mettre cette réponse en cache, forçant une nouvelle requête à chaque fois

Pragma: no-cache

Set-Cookie: DWcb100c0f811c1b6d691e3e7c406b56ba=deleted; expires=Thu, 01-Jan-1970 00:00:01 GMT; Max-Age=0; path=/wiki/; HttpOnly

Content-Encoding: gzip : Le serveur a compressé le corps de la réponse en utilisant gzip(conformément à ce que le client acceptait)

Keep-Alive: timeout=5, max=100

Connection: Keep-Alive

Transfer-Encoding: chunked : Le corps de la réponse n'a pas été envoyé en une seule fois, mais en plusieurs morceaux(chunks)

Content-Type: text/html; charset=utf-8 : Le contenu transféré est du code HTML, encodé en UTF-8

5. Faire un clic droit sur un échange HTTP puis Suivre ou Follow | HTTP Stream. Quelle différence ? Pourquoi ?

```
POST /wr2 HTTP/1.1
Host: o.pki.goog
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/ocsp-request
Content-Length: 84
Connection: keep-alive
Priority: u=2
Pragma: no-cache
Cache-Control: no-cache

0R0P0N0L0J0 ...+.....SB.....Mw|. #1.{5.....y..>7$.!..49mB.0....1..... ..I.r.
HTTP/1.1 200 OK
Content-Type: application/ocsp-response
Date: Sun, 14 Dec 2025 23:53:39 GMT
Cache-Control: public, max-age=14400
Content-Security-Policy-Report-Only: script-src 'none'; form-action 'none'; frame-src 'none'; r
port-uri https://csp.withgoogle.com/csp/scaffolding/sytroprc:52:0
Cross-Origin-Opener-Policy-Report-Only: same-origin; report-to=sytroprc:52:0
Report-To: {"group":"sytroprc:52:0","max_age":2592000,"endpoints":[{"url":"https://csp.withgoog
e.com/csp/report-to/scaffolding/sytroprc:52:0"}],}
Server: scaffolding on HTTPServer2
Content-Length: 472
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
```

-Requête

POST /wr2 HTTP/1.1 : Le client utilise la méthode POST pour envoyer les données à la ressource /wr2

Host: o.pki.goog : le serveur ciblé appartient à l'infrastructure de clé publique(PKI) de Google

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 : Le client est le navigateur Firefox 128.0

Accept: */*

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

Content-Type: application/ocsp-request : Point crucial. La requête est un message OCSP(Online Certificate Status Protocol), utilisé pour vérifier la révocation(validité) d'un certificat SSL/TLS

Content-Length: 84 : La taille du corps binaire de la requête OCSP est de 84 octets

Connection: keep-alive

Priority: u=2

Pragma: no-cache,

Cache-Control: no-cache : Le client insiste pour que la requête ne soit pas mise en cache, car la validité d'un certificat doit toujours être vérifiée avec les informations les plus fraîches.

-Réponse

HTTP/1.1 200 OK : Le serveur a traité la requête avec succès

Content-Type: application/ocsp-response : Confirme que la réponse contient le statut du certificat demandé

Date: Sun, 14 Dec 2025 23:53:39 GMT

Cache-Control: public, max-age=14400 : La réponse OCSP peut être mise en cache par tout intermédiaire ou le client(public) pendant 14400 secondes(4heures)

Content-Security-Policy-Report-Only: script-src 'none'; form-action 'none'; frame-src 'none'; report-uri <https://csp.withgoogle.com/csp/scaffolding/sytroprc:52:0>:Une politique de sécurité avancée qui surveille et rapproche les violations de la sécurité sans bloquer la page(mode rapport uniquement)

Cross-Origin-Opener-Policy-Report-Only: same-origin; report-to=sytroprc:52:0

Report-To: {"group":"sytroprc:52:0","max_age":2592000,"endpoints":[{"url":"<https://csp.withgoogle.com/csp/report-to/scaffolding/sytroprc:52:0>"}],}

Server: scaffolding on HTTPServer2 : Le logiciel ou la plateforme qui a servi la réponse

Content-Length: 472 : La taille du corps binaire de la réponse OCSP est de 472 octets

X-XSS-Protection: 0

X-Frame-Options: SAMEORIGIN : Protège contre le clickjacking en empêchant l'intégration de cette réponse dans une iframe d'un site différent

Port knocking

Le port knocking ('toquer à un port') est une méthode discrète permettant d'ouvrir à distance un port. Ce port est par défaut fermé, on ne peut pas y accéder, sauf si on toque avant à une série de ports fermés prédéfinis. On envoie des segments SYN TCP à une suite de ports prédéfinis pour ouvrir un port, c'est comme un code sécurité pour ouvrir une porte.

1. Trouver la séquence de ports utilisés pour accéder au port SSH de 172.17.123.177 dans la capture port-knocking.pcap disponible dans les données de ce TP3 sous Moodle

Si vous regardez un segment TCP sous Wireshark, vous pouvez voir les flags (drapeaux) TCP qui peuvent être filtrés avec le filtre flags (flags codés en hexa sur 12 bits - un nombre hexa code 4 bits)

```
1010 1111  Header Length: 40 bytes (10)
▼ Flags: 0x002 (SYN)
 000. .... = Reserved: Not set
...0 .... = Accurate ECN: Not set
.... 0... = Congestion Window Reduced: Not set
.... 0... = ECN-Echo: Not set
.... ..0. = Urgent: Not set
.... ...0 = Acknowledgment: Not set
.... .... 0... = Push: Not set
.... ..... 0... = Reset: Not set
▼ .... .... 1. = Syn: Set
  ► [Expert Info (Chat/Sequence): Connection establish request (SYN): server port 22]
.... .... 0... = Fin: Not set
```

Exemples :

tcp.flags == 0x002 pour syn
tcp.flags == 0x012 pour syn/ack
tcp.flags == 0x004 pour reset

Vous pouvez aussi regarder la valeur d'un drapeau indépendamment des autres avec ces filtres :

tcp.flags.syn == 1 pour SYN = 1
tcp.flags.ack == 0 pour ACK = 0

Si ouverte la communication SSH (port 22) (utilisant TCP) démarre par le 3-way handshake TCP :
SYN SYN/ACK ACK.



Synthèses



Protocole	Rôle principal
DNS	Résolution de noms de domaine
DHCP	Attribution d'adresses IP
SSH	Connexion distante sécurisée
FTP	Transfert de fichiers
HTTP	Transfert de pages web



Protocole	Port(s)	Sécurisé ?
DNS	53	NON (voir DNSSEC)
DHCP	67/68	NON
SSH	22	OUI
FTP	20/21	NON (voir FTPS/SFTP)
HTTP	80	NON (voir HTTPS port 443)



Protocole	Encapsulation sur Ethernet			
DNS	ETH	IP	UDP	DNS
DHCP	ETH	IP	UDP	DHCP
SSH	ETH	IP	TCP	SSH
FTP	ETH	IP	TCP	FTP
HTTP	ETH	IP	TCP	HTTP



Protocole	Commande utile
DNS	nslookup
DHCP	dhclient
SSH	ssh
FTP	ftp
HTTP	un navigateur, curl, wget